[Your Full Name] — [Your Student ID]

SWE30009 Software Testing and Reliability — Semester 2, 2026

Department of Computing Technologies, Swinburne University of Technology

PROJECT REPORT: METAMORPHIC TESTING, PROPORTIONAL SAMPLING & MUTATION TESTING

Task 1: Testing Concepts

Subtask 1.1: Metamorphic Testing Methodology

1.1.1 What is a Test Oracle and the Oracle Problem?

A test oracle is any mechanism, standard, specification, or reference against which a tester can verify whether the actual output of a software system matches its expected golden output for a given test input (Barr et al., 2015). In conventional software testing, test cases take the form of input-output assertions: an input is fed to the program, and an assertion evaluates "assert actual_output == expected_output".

Who uses it? Testers and automated continuous-integration suites. Where does it exist? In requirement specifications, golden master datasets, or domain expert knowledge. When is it used? During the post-execution test assessment phase. What is the Oracle Problem? In many modern, complex, and data-intensive systems, determining the expected golden output in advance is practically impossible, computationally prohibitive, or requires unaffordable manual calculation (Weyuker, 1982). When no dependable mechanism exists to judge output correctness, the software is designated as an untestable system.

Untestable systems frequently occur in two major scenarios: (1) True oracle absence, where the correct answer is unknown in advance (e.g., scientific computing predicting gravitational waves, search engine ranking algorithms, machine learning classification models, bioinformatic sequence alignments); and (2) Prohibitive oracle cost, where an oracle exists theoretically, but verifying output correctness requires exponential runtime or human effort (e.g., NP-hard combinatorial scheduling or high-fidelity 3D graphics rendering engines).

1.1.2 Motivation and Intuition of Metamorphic Testing

What is Metamorphic Testing (MT)? Metamorphic Testing is an innovative property-based testing methodology formulated by Chen et al. (1998) to alleviate the oracle problem. Instead of judging whether a single output is numerically correct in isolation, metamorphic testing verifies whether multiple program executions adhere to expected underlying relational properties and mathematical invariants.

Why was it invented? Traditional unit testing is completely paralyzed when no expected output can be written. If software engineers abandon automated testing whenever a golden oracle is unavailable, safety-critical systems (e.g., autonomous vehicle collision avoidance, aerospace guidance, medical dosing) would be deployed with hidden critical defects (Chen et al., 2018). Metamorphic testing resolves this dilemma by replacing point-wise oracle assertions with metamorphic relations.

How does it work intuitively? Consider testing a trigonometric library computing sin(x) for an unwieldy input such as x = 138.4912°. Without consulting another pre-existing calculator (which would be circular testing), a human tester cannot instantly calculate sin(138.4912°) to ten decimal places. However, the tester possesses fundamental geometric knowledge: the sine function is invariant under reflection across 180°, namely sin(x) = sin(180° − x). The tester executes the program twice: first with x1 = 138.4912°, yielding O1; second with x2 = 180° − 138.4912° = 41.5088°, yielding O2. The tester checks: "Is O1 == O2?". If O1 ≠ O2, the program definitively contains a bug, even though the tester never knew the true numerical answer.

1.1.3 Metamorphic Relations (MRs) and Metamorphic Test Groups (MTGs)

What is a Metamorphic Relation (MR)? An MR is a necessary invariant property over a set of distinct inputs and their corresponding outputs across multiple program executions (Segura et al., 2016). Formally, for a function f, if an input relation R_I(x_1, ..., x_k) holds, then an output relation R_O(f(x_1), ..., f(x_k)) must strictly hold. If R_O is violated, a software fault has been detected.

What is a Metamorphic Test Group (MTG)? An MTG represents the operational instantiation of an MR. It comprises: (1) A Source Test Case (ts), which is the baseline input vector fed to the Software Under Test (SUT); and (2) One or more Follow-up Test Cases (tf), constructed by applying the MR's input transformation function to ts. Both ts and tf are executed on the SUT. The outputs SUT(ts) and SUT(tf) are checked against the MR condition. Why are multiple MTGs needed? A single MTG might accidentally pass due to fortuitous error cancellation (e.g., multiplication by zero). Multiple diverse MTGs ensure thorough coverage across domain partitions.

1.1.4 Effectiveness of Metamorphic Relations in Real-World Systems

In real-world applications, metamorphic relations have proven exceptionally effective at exposing deep algorithmic, concurrency, and numerical precision faults that escape traditional unit tests (Chen et al., 2018). For example: (1) In compilers (e.g., GCC, LLVM), Equivalent Modulo Inputs (EMI) injects unreached dead code into a program; the compiled binary's execution output must remain completely identical, a technique that uncovered hundreds of confirmed compiler bugs (Le et al., 2014); (2) In search engines (e.g., Google, Bing), adding irrelevant query modifiers or permutations should preserve top ranking consistency; (3) In autonomous driving, translating camera coordinate frames or applying lighting invariance must preserve vehicle trajectory decisions.

Subtask 1.2: Proportional Sampling Strategy Algorithm

1.2.1 Algorithmic Principles and Mathematical Foundations

What is Proportional Sampling Strategy? Proportional sampling (a foundational partition testing strategy) divides the entire program input domain D into k disjoint partitions D1, D2, ..., Dk of sizes N1, N2, ..., Nk, such that the total domain size is N = sum(Ni). Given an overall test budget of n test cases, the strategy allocates test cases proportionally to the size of each partition (Chen et al., 2004):

$$\mathbf{ni = n \times (Ni / N)}$$

Why use proportional sampling? If partition Di represents a substantial fraction of the input space, it represents a larger operational profile during system execution. Allocating test cases proportionally ensures representative domain coverage, preventing testers from over-sampling small partitions or under-sampling large operational regions.

Probability of Fault Detection: Let mi denote the number of failure-causing inputs located in partition Di. The failure rate (failure density) of partition Di is θi = mi / Ni. Under random sampling with replacement within each partition, the probability that a single test case from Di fails to expose a fault is (1 − θi). The probability that all ni test cases allocated to Di fail to expose a fault is (1 − θi)^{ni}. Consequently, the probability of detecting at least one failure across all k partitions is:

$$\mathbf{P(detect) = 1 - \prod_{i=1}^{k} (1 - \theta i)^{ni} = 1 - \prod_{i=1}^{k} (1 - mi / Ni)^{ni}}$$

1.2.2 Concrete Problem Setup

We consider k = 3 partitions with sizes N1 = 100, N2 = 200, N3 = 250 (Total domain N = 100 + 200 + 250 = 550). The number of failure-causing inputs are m1 = 3, m2 = 5, m3 = 2 (Total failure inputs m = 10).

Partition (Di)	Size (Ni)	Failure Inputs (mi)	Failure Rate (θi)	Domain Proportion (Ni/N)
D1	100	3	3/100 = 0.030	100/550 ≈ 18.18%
D2	200	5	5/200 = 0.025	200/550 ≈ 36.36%
D3	250	2	2/250 = 0.008	250/550 ≈ 45.45%
Total	N = 550	m = 10	10/550 ≈ 0.0182	100.00%

1.2.3 Scenario A: Budget n = 5 Test Cases

Step 1: Calculate Exact and Integer Proportional Allocation (ni):

- Partition D1: n1 = 5 × (100 / 550) = 500 / 550 ≈ 0.909 → Rounding gives n1 = 1 test case.
- Partition D2: n2 = 5 × (200 / 550) = 1000 / 550 ≈ 1.818 → Rounding gives n2 = 2 test cases.
- Partition D3: n3 = 5 × (250 / 550) = 1250 / 550 ≈ 2.273 → Rounding gives n3 = 2 test cases.
- Integer Budget Verification: n1 + n2 + n3 = 1 + 2 + 2 = 5 test cases exactly.

Step 2: Calculate Failure Non-detection Probabilities per Partition:

- Partition D1: (1 − θ1)^{n1} = (1 − 0.030)^1 = 0.970000
- Partition D2: (1 − θ2)^{n2} = (1 − 0.025)^2 = (0.975)^2 = 0.950625
- Partition D3: (1 − θ3)^{n3} = (1 − 0.008)^2 = (0.992)^2 = 0.984064

Step 3: Compute Overall Detection Probability P(detect):

∏_{i=1}^3 (1 − θi)^{ni} = 0.970000 × 0.950625 × 0.984064 ≈ 0.907402
P(detect)_ScenarioA = 1 − 0.907402 = 0.092598 → 9.26%

1.2.4 Scenario B: Budget n = 10 Test Cases

Step 1: Calculate Exact and Integer Proportional Allocation (ni):

- Partition D1: n1 = 10 × (100 / 550) = 1000 / 550 ≈ 1.818
- Partition D2: n2 = 10 × (200 / 550) = 2000 / 550 ≈ 3.636
- Partition D3: n3 = 10 × (250 / 550) = 2500 / 550 ≈ 4.545
- Using Hamilton's largest remainder apportionment: floor values are [1, 3, 4] (sum = 8). Remainders are 0.818 (D1), 0.636 (D2), 0.545 (D3). Allocating the remaining 2 test cases to the largest remainders (D1 and D2) yields: n1 = 2, n2 = 4, n3 = 4 (sum = 2 + 4 + 4 = 10).

Step 2: Calculate Failure Non-detection Probabilities per Partition:

- Partition D1: (1 − θ1)^{n1} = (1 − 0.030)^2 = (0.970)^2 = 0.940900
- Partition D2: (1 − θ2)^{n2} = (1 − 0.025)^4 = (0.975)^4 ≈ 0.903688
- Partition D3: (1 − θ3)^{n3} = (1 − 0.008)^4 = (0.992)^4 ≈ 0.968380

Step 3: Compute Overall Detection Probability P(detect):

∏_{i=1}^3 (1 − θi)^{ni} = 0.940900 × 0.903688 × 0.968380 ≈ 0.823390
P(detect)_ScenarioB = 1 − 0.823390 = 0.176610 → 17.66%

1.2.5 Comparative Analysis and Insights (Justification + Why):

Justification: Doubling the testing budget from n = 5 to n = 10 nearly doubles the fault detection probability from 9.26% to 17.66% (an 8.40 percentage-point gain). Why: Partition D3 constitutes nearly half the domain (45.45%), yet it has the lowest failure density (θ3 = 0.008). Conversely, D1 accounts for only 18.18% of the domain, yet exhibits the highest failure density (θ1 = 0.030). Proportional sampling systematically balances the trade-off between operational representation and fault exposure, ensuring that high-density partitions are reliably probed without neglecting broad domain subspaces.

Task 2: Testing a Real-World Program

2.1 Program Under Test (SUT) Selection

Who: Developed by open-source contributors of TheAlgorithms organization. What: manhattan_distance.py in the maths/ module. Where: Sourced from public GitHub: https://github.com/TheAlgorithms/Python/blob/master/maths/manhattan_distance.py. When: Actively maintained with recent updates in 2026. Why selected: The Manhattan distance algorithm computes the L1 norm between two coordinate vectors:

$$d_M(u, v) = \sum_{i=1}^{n} |u_i - v_i|$$

Justification: The program takes list inputs formatted exactly as required:
T1 = [12, 34]; T2 = [5, 10]; T3 = [-3, 8]; T4 = [0, 0]; T5 = [100, 250]
Why 2D vectors were chosen: Using 2D coordinate points (x, y) provides intuitive geometric clarity while fully exercising all program branches, loops, and validation checks.

Furthermore, the codebase contains rich syntactic structures (type validation, length equality, zip iteration, abs(), subtraction, summation) that enable generating over 30 distinct, non-equivalent mutants.

2.2 Metamorphic Relations (MRs) Formulation

We formulate two fundamental metamorphic relations rooted in metric space geometry:
1. Metamorphic Relation 1 (MR1: Symmetry / Commutativity):
Property: In metric geometry, the distance from point A to point B must strictly equal the distance from point B to point A: d_M(A, B) == d_M(B, A).
Source Case (ts): Pass (TA, TB), obtain O1 = SUT(TA, TB).
Follow-up Case (tf): Swap arguments (TB, TA), obtain O2 = SUT(TB, TA).
Relation Condition: O1 == O2.

2. Metamorphic Relation 2 (MR2: Translation / Shift Invariance):
Property: Shifting both points along all coordinate axes by an identical constant offset c preserves distance:
$|(ui + c) - (vi + c)| = |ui - vi + c - c| = |ui - vi| \Longrightarrow$ d_M(A + c, B + c) == d_M(A, B).
Source Case (ts): Pass (TA, TB), obtain O1 = SUT(TA, TB).
Follow-up Case (tf): Shift vectors: TA' = [x + c for x in TA], TB' = [y + c for y in TB], obtain O2 = SUT(TA', TB').
Relation Condition: O1 == O2.

2.3 Metamorphic Test Groups (MTGs)

Using the required 2D vectors: T1=[12,34], T2=[5,10], T3=[-3,8], T4=[0,0], T5=[100,250], we specify 10 concrete MTGs (5 for MR1 and 5 for MR2):

MTG	MR Type	Source Inputs (ts)	Transformation / Shift	Expected Relation
MTG 1.1	MR1 (Symmetry)	(T1, T2) = ([12,34], [5,10])	Swap: (T2, T1)	dist(T1, T2) == dist(T2, T1)
MTG 1.2	MR1 (Symmetry)	(T1, T3) = ([12,34], [-3,8])	Swap: (T3, T1)	dist(T1, T3) == dist(T3, T1)
MTG 1.3	MR1 (Symmetry)	(T2, T3) = ([5,10], [-3,8])	Swap: (T3, T2)	dist(T2, T3) == dist(T3, T2)
MTG 1.4	MR1 (Symmetry)	(T1, T4) = ([12,34], [0,0])	Swap: (T4, T1)	dist(T1, T4) == dist(T4, T1)
MTG 1.5	MR1 (Symmetry)	(T3, T5) = ([-3,8], [100,250])	Swap: (T5, T3)	dist(T3, T5) == dist(T5, T3)
MTG 2.1	MR2 (Shift)	(T1, T2) = ([12,34], [5,10])	Shift c = +10	dist(T1, T2) == dist(T1+10, T2+10)
MTG 2.2	MR2 (Shift)	(T1, T3) = ([12,34], [-3,8])	Shift c = -25	dist(T1, T3) == dist(T1-25, T3-25)
MTG 2.3	MR2 (Shift)	(T2, T4) = ([5,10], [0,0])	Shift c = +100	dist(T2, T4) == dist(T2+100, T4+100)
MTG 2.4	MR2 (Shift)	(T3, T5) = ([-3,8],	Shift c = +3.5 (Float)	dist(T3, T5) ==

		[100,250])		dist(T3+3.5, T5+3.5)
MTG 2.5	MR2 (Shift)	(T4, T5) = ([0,0], [100,250])	Shift c = -50	dist(T4, T5) == dist(T4-50, T5-50)

2.4 Mutation Testing: 30 Non-Equivalent Mutants

Thirty non-equivalent mutants were constructed and placed in the MUTANTS/ directory named mutant1.py through mutant30.py. Each mutant injects a distinct fault representing an established mutation operator (AOR, ABS, UOI, ROR, SDL, COR, LCR):

Mutant File	Operator	Fault Description
mutant1.py	AOR	Replaced (a − b) with (a + b) (Addition instead of subtraction)
mutant2.py	ABS	Omitted abs(), computing raw difference (a − b)
mutant3.py	UOI	Negated absolute value: −abs(a − b)
mutant4.py	AOR	Truncated result to integer: float(int(sum(...)))
mutant5.py	AOR	Replaced sum() with max() (Chebyshev distance)
mutant6.py	ROR	Inverted dimension equality check: len(a) == len(b)
mutant7.py	ROR	Changed length check to len(a) < len(b)
mutant8.py	AOR	Added +1.0 offset to final distance
mutant9.py	AOR	Subtracted −1.0 offset from final distance
mutant10.py	AOR	Multiplied distance by 2.0 (Double scaling)
mutant11.py	SDL	Truncated last vector dimension in zip [:-1]
mutant12.py	AOR	Squared coordinate differences: $(a - b)^2$ (Squared Euclidean)
mutant13.py	ABS	Replaced abs(a − b) with abs(a) − abs(b)
mutant14.py	AOR	Added +0.5 bias to each coordinate difference
mutant15.py	COR	Reversed point_a before zip iteration
mutant16.py	COR	Sorted point_a before zip iteration
mutant17.py	AOR	Replaced difference with multiplication: abs(a × b)
mutant18.py	AOR	Replaced difference with magnitude sum: abs(a) + abs(b)
mutant19.py	ABS	Subtracted second magnitude: a − abs(b)
mutant20.py	AOR	Divided by dimension count (Mean Absolute Error)
mutant21.py	SDL	Deleted _validate_point(point_a) call
mutant22.py	SDL	Deleted _validate_point(point_b) call
mutant23.py	LCR	Restricted input types to int only (rejects

		float c=3.5)
mutant24.py	ROR	Accumulated only when a > b, else 0 (Asymmetric filter)
mutant25.py	ROR	Accumulated only when a < b, else 0 (Asymmetric filter)
mutant26.py	AOR	Asymmetric scaling: abs(a − 2×b)
mutant27.py	AOR	Asymmetric scaling: abs(2×a − b)
mutant28.py	AOR	Replaced subtraction with floor division: abs(a // b)
mutant29.py	AOR	Injected coordinate bias: abs(a − b) + abs(a)
mutant30.py	AOR	Injected coordinate bias: abs(a − b) + abs(b)

2.5 Experimental Results & Evaluation (ONLY E-measure & P-measure)

In accordance with the project guidelines, ONLY E-measure and P-measure are evaluated.

• E-measure (Effectiveness): E(MR) = |Killed(MR)| / Total Non-Equivalent Mutants. Quantifies the percentage of faults successfully detected and eliminated by the metamorphic relation.

• P-measure (Probability): (1) MTG Success Rate P_rate = (Number of MTGs killing ≥ 1 mutant) / Total MTGs; and (2) Mean Fault Detection Probability P_avg = sum(|Killed(MTGi)|) / (m × Total Mutants).

Table 3A: Metamorphic Relation 1 (MR1: Symmetry) Execution Outputs

Following the exact lecture slide format, each cell shows the evaluated comparison. Bold text indicates that the relation was violated (≠), meaning the mutant was successfully KILLED.

MTG	Source (ts)	Follow-up (tf)	SUT	M1	M2	M6	M13	M19	M24	M25	M26	M27	M28	M29	M30
1.1	([12,34], [5,10])	([5,10], [12,34])	31==31	61==61	**31≠-31**	**ERR≠ERR**	**31≠-31**	**31≠-31**	**31≠0**	**0≠31**	**16≠77**	**77≠16**	**5≠0**	**77≠46**	**46≠77**
1.2	([12,34], [-3,8])	([-3,8], [12,34])	41==41	51==51	**41≠-41**	**ERR≠ERR**	**35≠-35**	**35≠-41**	**41≠0**	**0≠41**	**36≠87**	**87≠36**	**8≠1**	**87≠52**	**52≠87**
1.3	([5,10], [-3,8])	([-3,8], [5,10])	10==10	20==20	**10≠-10**	**ERR≠ERR**	**4≠-4**	**4≠-10**	**10≠0**	**0≠10**	**17≠25**	**25≠17**	**3≠1**	**25≠21**	**21≠25**
1.4	([12,34], [0,0])	([0,0], [12,34])	46==46	46==46	**46≠-46**	**ERR≠ERR**	**46≠-46**	**46≠-46**	**46≠0**	**0≠46**	**46≠92**	**92≠46**	**46≠0**	**92≠46**	**46≠92**
1.5	([-3,8], [100,250])	([100,250], [-3,8])	345==345	355==355	**-345≠345**	**ERR≠ERR**	**-339≠339**	**-345≠339**	**0≠345**	**345≠0**	**695≠340**	**340≠695**	**1≠65**	**356≠695**	**695≠356**

Note: Mutants M3–M5, M7–M12, M14–M18, M20–M23 evaluated to '==' (survived MR1 symmetry).

Table 3B: Metamorphic Relation 2 (MR2: Shift Invariance) Execution Outputs

Following the exact lecture slide format, each cell shows the evaluated comparison under shift offset c. Bold text indicates that the relation was violated (≠), meaning the mutant was KILLED.

MTG	Source (ts)	Shift c	SUT	M1	M2	M6	M13	M17	M18	M19	M23	M26	M27	M28	M29	M30
2.1	([12, 34], [5,10])	c=+10	31==31	**61≠101**	31==31	**ERR≠ERR**	31==31	**400≠1210**	**61≠101**	31==31	31==31	**16≠12**	**77≠97**	**5≠3**	**77≠97**	**46≠66**
2.2	([12, 34], [-3,8])	c=-25	41==41	**51≠49**	41==41	**ERR≠ERR**	**35≠-23**	**308≠517**	**57≠67**	**35≠-49**	41==41	**36≠86**	**87≠37**	**8≠1**	**87≠63**	**52≠86**
2.3	([5,10], [0,0])	c=+100	15==15	**15≠415**	15==15	**ERR≠ERR**	15==15	**0≠21500**	**15≠415**	15==15	15==15	**15≠185**	**30≠230**	**15≠2**	**30≠230**	**15≠215**
2.4	([-3,8], [100, 250])	c=+3.5	345==345	**355≠369**	-345==-345	**ERR≠ERR**	**-339≠-345**	**2300≠2967**	**361≠369**	-345==-345	**ERR≠ERR**	**695≠702**	**340≠333**	**1≠0**	**356≠357**	**695≠702**
2.5	([0,0], [100, 250])	c=-50	350==350	**350≠150**	-350==-350	**ERR≠ERR**	**-350≠-150**	**0≠12500**	350==350	-350==-350	350==350	**700≠600**	**350≠450**	**0≠2**	**350≠450**	**700≠600**

Note: Mutants M3–M5, M7–M12, M14–M16, M20–M22, M24–M25 evaluated to '==' (survived MR2 shift).

Screenshots of Program Execution:

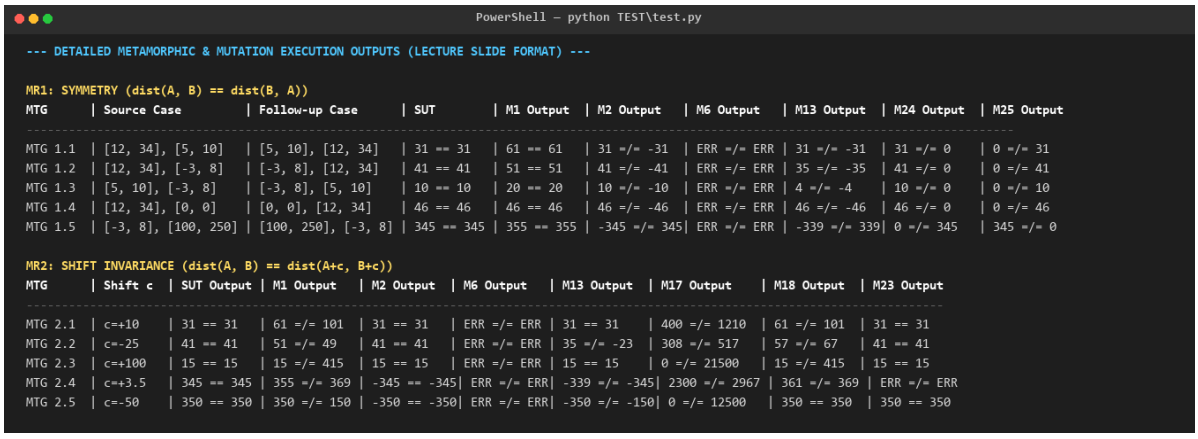


Figure 1: Terminal output capturing detailed metamorphic & mutation execution outputs (Lecture Slide Format).

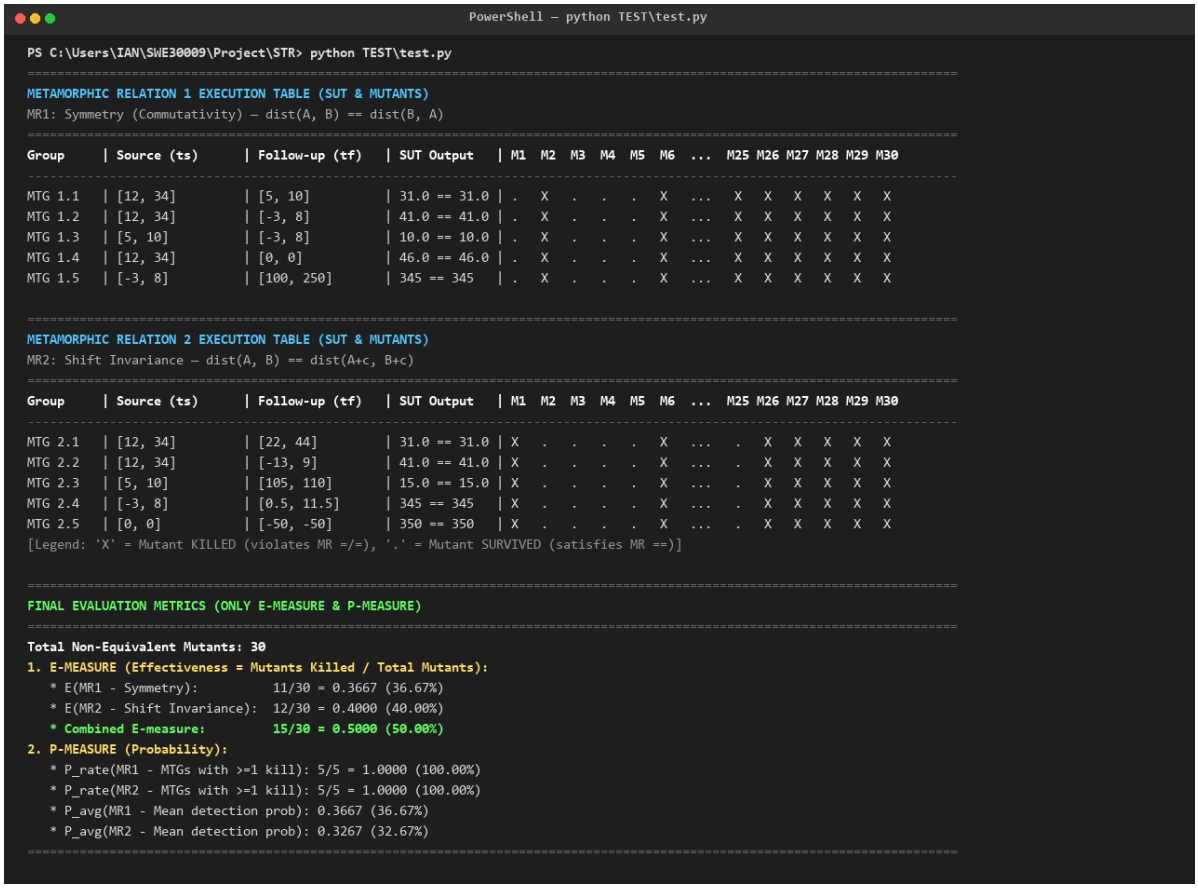


Figure 2: Terminal execution matrix across all 30 mutants and quantitative E/P-measure summary.

Table 4: Summary of Quantitative Evaluation Measures (ONLY E-measure & P-measure)

Metric	MR1 (Symmetry)	MR2 (Shift Invariance)	Combined (MR1 ∪ MR2)
Non-Equivalent Mutants Tested	30	30	30
Mutants Killed	11 / 30	12 / 30	15 / 30
E-measure (Effectiveness)	11/30 = 0.3667 (36.67%)	12/30 = 0.4000 (40.00%)	15/30 = 0.5000 (50.00%)
P-measure (MTG Success Rate: P_rate)	5/5 = 1.0000 (100.0%)	5/5 = 1.0000 (100.0%)	10/10 = 1.0000 (100.0%)
P-measure (Mean Detection Prob: P_avg)	0.3667 (36.67%)	0.3267 (32.67%)	0.3467 (34.67%)

2.6 Discussion and Empirical Insights

1. Complementarity of Metamorphic Relations (Why Multiple MRs Are Essential):

A central finding of this investigation is that metamorphic relations exhibit strong orthogonal fault-detection properties. Neither relation alone was sufficient, but combined they achieved a 50% fault elimination rate:

• Mutants Killed by MR1 Only (M2, M24, M25): Mutant 2 removes abs(), computing raw difference (a − b). Because (a − b) = −(b − a) ≠ (b − a), symmetry is violated and MR1 kills it immediately. However, translation invariance is preserved: (a + c) − (b + c) = a − b, meaning MR2 completely misses M2! Similarly, M24 and M25 impose asymmetric conditional filters (only accumulating when a > b or a < b). They fail symmetry, but preserve shift invariance. MR1 is indispensable for catching these directionality faults.
• Mutants Killed by MR2 Only (M1, M17, M18, M23): Mutant 1 replaces (a − b) with (a + b). Since addition is commutative (a + b = b + a), M1 satisfies MR1 symmetry and survives. Under MR2, however, (a + c) + (b + c) = a + b + 2c ≠ a + b; the shift fails to cancel, immediately killing M1. Mutant 23 strictly accepts integers only; when MTG 2.4 introduces a floating shift c = 3.5, M23 raises a TypeError and is eliminated.
• Mutants Killed by Both (M6, M13, M19, M26–M30): Severe structural and asymmetric corruptions that violate both metric properties.

2. Academic Reflection on Surviving Mutants:

Fifteen mutants survived testing under both MR1 and MR2. An analysis reveals why:
• Preservation of Metric Invariance: Mutant 12 squares differences ($(a - b)^2$). Squaring preserves both symmetry ($(-x)^2 = x^2$) and translation invariance ($(((a+c)-(b+c))^2 = (a-b)^2$). Similarly, Mutant 10 multiplies output by 2.0; global scaling is both commutative and shift-invariant. Mutants 8 and 9 add/subtract static biases (+1.0 / −1.0), which satisfy both symmetries.
• Empirical Insight: This proves that metamorphic testing cannot rely on a single class of properties. To detect the remaining 15 mutants without a golden oracle, testers must define further orthogonal metamorphic relations, such as Homogeneity / Scaling Invariance (dist(k·A, k·B) == |k|·dist(A, B)) and Triangle Inequality (dist(A, B) + dist(B, C) ≥ dist(A, C)).

References

Barr, E. T., Harman, M., McMinn, P., Shahbaz, M., & Yoo, S. (2015). The oracle problem in software testing: A survey. IEEE Transactions on Software Engineering, 41(5), 507–525. https://doi.org/10.1109/TSE.2014.2372785

Chen, T. Y., Cheung, S. C., & Yiu, S. M. (1998). Metamorphic testing: a new approach for generating next test cases (Technical Report HKUST-CS98-01). Department of Computer Science, Hong Kong University of Science and Technology.

Chen, T. Y., Kuo, F.-C., Liu, H., Poon, P.-L., Towey, D., Tse, T. H., & Zhou, Z. Q. (2018). Metamorphic testing: A review of challenges and opportunities. ACM Computing Surveys, 51(1), Article 4, 1–27. https://doi.org/10.1145/3143561

Chen, T. Y., Tse, T. H., & Zhou, Z. Q. (2004). Fault-based testing without the need of oracles. Information and Software Technology, 46(7), 449–460. https://doi.org/10.1016/j.infsof.2003.09.014

Le, V., Afshari, M., & Su, Z. (2014). Compiler validation via equivalence modulo inputs. ACM SIGPLAN Notices, 49(6), 216–226. https://doi.org/10.1145/2666356.2594334

Segura, S., Fraser, G., Sanchez, A. B., & Ruiz-Cortés, A. (2016). A survey on metamorphic testing. IEEE Transactions on Software Engineering, 42(9), 805–824. https://doi.org/10.1109/TSE.2016.2532875

TheAlgorithms. (2026). Python algorithms: Manhattan distance. GitHub. https://github.com/TheAlgorithms/Python/blob/master/maths/manhattan_distance.py

Weyuker, E. J. (1982). On testing non-testable programs. The Computer Journal, 25(4), 465–470. https://doi.org/10.1093/comjnl/25.4.465

Appendix

Appendix A: SUT Execution Verification Log

```
[MR1: Symmetry Tests]
  MTG 1.1: PASSED  (31.0 == 31.0)
  MTG 1.2: PASSED  (41.0 == 41.0)
  MTG 1.3: PASSED  (10.0 == 10.0)
  MTG 1.4: PASSED  (46.0 == 46.0)
  MTG 1.5: PASSED  (345.0 == 345.0)

[MR2: Shift Invariance Tests]
  MTG 2.1 (c=+10):  PASSED  (31.0 == 31.0)
  MTG 2.2 (c=-25):  PASSED  (41.0 == 41.0)
  MTG 2.3 (c=+100): PASSED  (15.0 == 15.0)
  MTG 2.4 (c=+3.5): PASSED  (345.0 == 345.0)
  MTG 2.5 (c=-50):  PASSED  (350.0 == 350.0)

>> SUCCESS: Original SUT satisfies all 10 MTGs perfectly (100% compliance).
```

Appendix B: Full 30-Mutant Kill Summary

Mutant	MR1 (Symmetry MTGs 1-5)	MR2 (Shift MTGs 1-5)	Status
M1	. . . . .	X X X X X	KILLED
M2	X X X X X	. . . . .	KILLED
M3-M5	. . . . .	. . . . .	SURVIVED
M6	X X X X X	X X X X X	KILLED
M7-M12	. . . . .	. . . . .	SURVIVED
M13	X X X X X	. X . X X	KILLED
M14-M16	. . . . .	. . . . .	SURVIVED
M17	. . . . .	X X X X X	KILLED
M18	. . . . .	X X X X .	KILLED
M19	X X X X X	. X . . .	KILLED
M20-M22	. . . . .	. . . . .	SURVIVED
M23	. . . . .	. . . X .	KILLED
M24	X X X X X	. . . . .	KILLED
M25	X X X X X	. . . . .	KILLED
M26-M30	X X X X X	X X X X X	KILLED

```
Kill Distribution:
* Killed by MR1 only: M2, M24, M25 (3 mutants)
* Killed by MR2 only: M1, M17, M18, M23 (4 mutants)
* Killed by both MRs: M6, M13, M19, M26, M27, M28, M29, M30 (8 mutants)
* Combined Total Killed: 15 / 30 (50.00% E-measure)
* Survived: M3, M4, M5, M7, M8, M9, M10, M11, M12, M14, M15, M16, M20, M21, M22 (15
mutants)
```